

LocalMesh

Private Network Service Platform

Phase 1 report

Build and observe the network

Course	Computer Networks
Review	Phase 1 / Review 1
Zone	localmesh.test
Authors	Harsith (DNS, UTM host) Manohar (nginx, TLS, edge)
IPs	10.7.12.104 10.7.2.38

The application stays simple — the network is the project.

Contents

1 Abstract	2
2 Purpose and constraints	2
3 Architecture	2
3.1 Roles versus hardware	2
3.2 LAN topology	3
3.3 Request path	3
3.4 UTM publication	3
4 Protocol layers	4
5 Implementation (what we built, not a runbook)	5
5.1 Private DNS	5
5.2 Backends	5
5.3 Edge	5
5.4 TLS	5
6 Observations and evidence	5
7 Failure analysis	6
8 Learning notes (viva)	6
9 What is not in this repository	6
10 Conclusion	6
A Reference configuration	6
A.1 dnsmasq (excerpt)	7
A.2 nginx upstream (excerpt)	7
A.3 Backend contract	7

1 Abstract

LocalMesh is a fully local private service: a client types `https://app.localmesh.test:8443`, resolves the name through the team's own DNS, completes TLS at a reverse proxy, and receives JSON from one of two backend processes. Phase 1 implements four network roles on two physical Macs. Your Mac (10.7.12.104) runs `dnsmasq` and hosts two Ubuntu guests in UTM Emulated VLAN. Manohar's Mac (10.7.2.38) terminates TLS and load-balances to 10.7.12.104:3001 and :3002. This report records the architecture, protocol mapping, configuration, observations, and failure analysis required for Review 1. It is the commitable Phase 1 document. Machine-by-machine setup runbooks stay out of git.

2 Purpose and constraints

The brief asks for a private environment with no cloud. A client must (i) use a `.test` name, (ii) hit the team DNS, (iii) open HTTPS on the team edge, (iv) see both backends through the load balancer, and (v) show the path in packet captures. Ports 8080/8443 may replace 80/443. Teams of two or three may combine hardware as long as the four *roles* exist.

We chose Option A: both backends on Your Mac. That is allowed. Emulated VLAN plus port forwarding is a hypervisor feature; it is not “implementing NAT” as a course topic. Phase 1 explicitly does not require student-built NAT, VLANs, or routing protocols.

3 Architecture

3.1 Roles versus hardware

Table 1: Mapping from the brief to this team.

Brief	This team	LAN identity	Software
Mac 1 DNS + client	Your Mac	10.7.12.104	<code>dnsmasq :53</code>
Mac 2 edge + TLS + LB	Manohar's Mac	10.7.2.38	<code>nginx :8080/:8443</code>
Mac 3 Backend A	UTM VM, forward 3001	10.7.12.104:3001	Python A
Mac 4 Backend B + client	UTM LocalMesh-1, forward 3002	10.7.12.104:3002	Python B

Shared settings live only on each machine in `~/cn-team.env` (`TEAM=localmesh`). They are not committed.

3.2 LAN topology

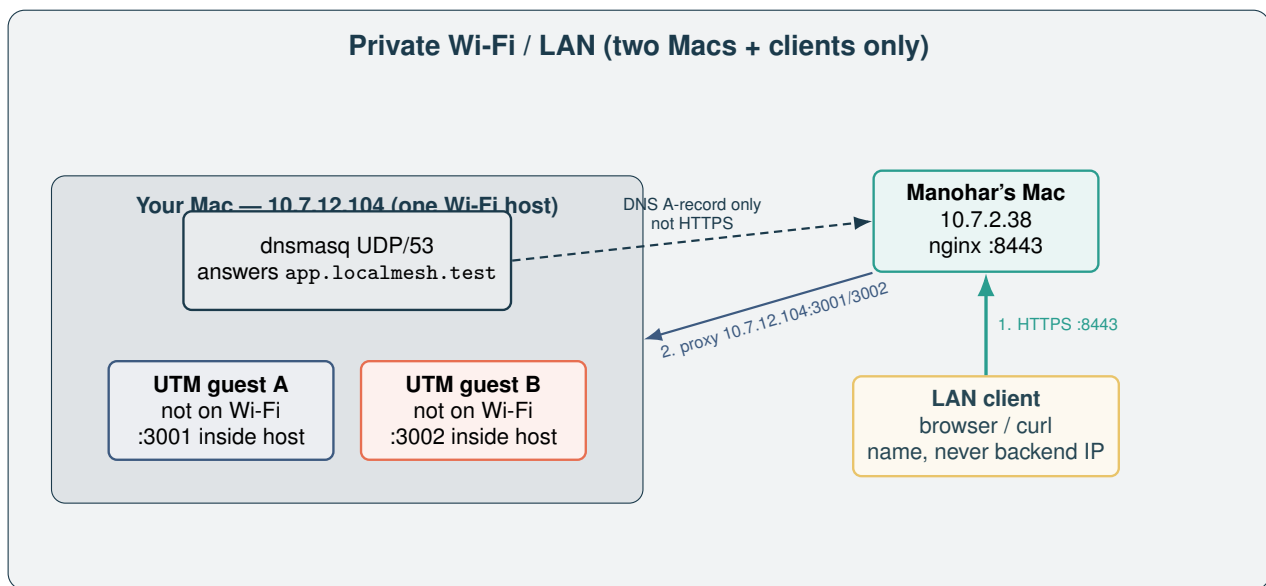


Figure 1: Wi-Fi has two Macs. Backends are UTM guests inside Your Mac, not LAN hosts. Dashed arrow = DNS A-record only. HTTPS is always client → Manohar :8443, then proxy to 10.7.12.104:3001 or :3002.

Clients always use the name. nginx is the only process they should treat as “the service”. The dashed Your-Mac-to-Manohar arrow is **not** the HTTPS path; it is the DNS answer `app.localmesh.test` → 10.7.2.38. Browsers then open TCP to Manohar :8443. Guests sit on UTM’s user-mode VLAN (classically 10.0.2.0/24). They are not extra Wi-Fi IPs. Manohar cannot ping those addresses. He reaches backends only as 10.7.12.104:3001 and :3002.

3.3 Request path

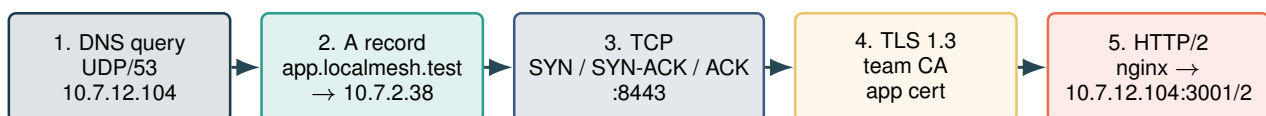


Figure 2: One successful client request, left to right.

3.4 UTM publication

UTM Shared Network never shows a Port Forward table. Emulated VLAN does. Host Address must be the Wi-Fi IPv4, not blank (blank binds loopback only).

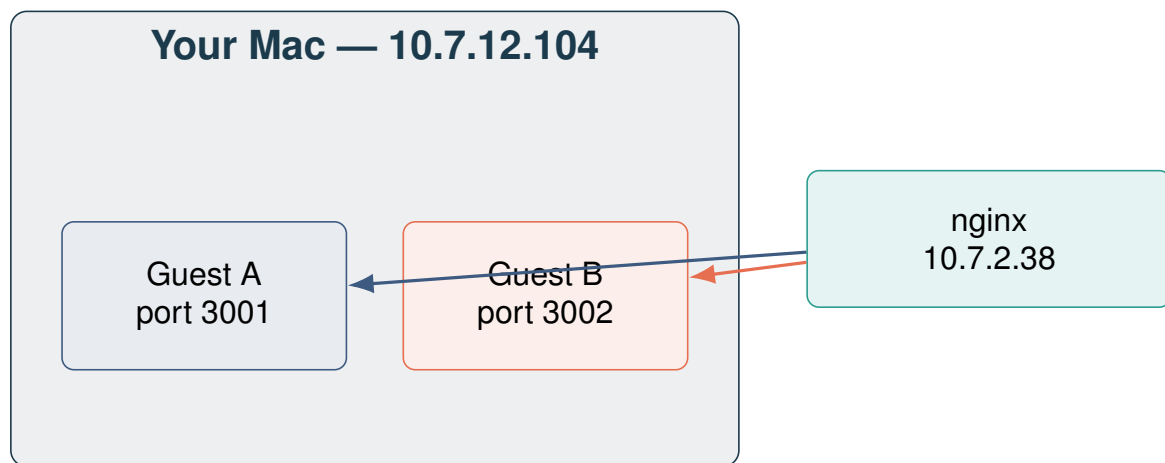


Figure 3: nginx talks only to Your Mac's published ports.

4 Protocol layers

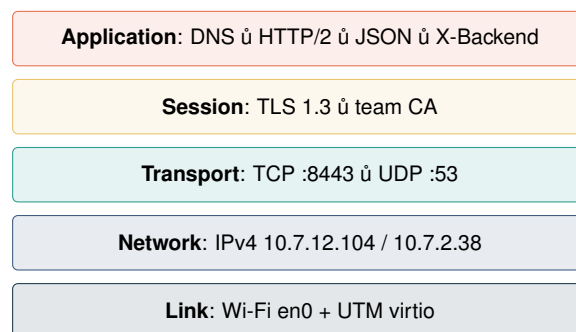


Figure 4: Classroom stack mapped onto LocalMesh.

DNS. dnsmasq listens on 127.0.0.1 and 10.7.12.104. `local=/localmesh.test/` means that zone is never forwarded. `host-record` creates A and PTR for `app` and `api` to 10.7.2.38. Other names go to 8.8.8.8 and 1.1.1.1. Evidence: `dig SERVER 10.7.12.104#53`, answer 10.7.2.38.

TCP. After resolution the client opens 10.7.2.38:8443. Capture the SYN, SYN-ACK, ACK before any application data. Source port is ephemeral; destination is 8443 (or 53 for DNS).

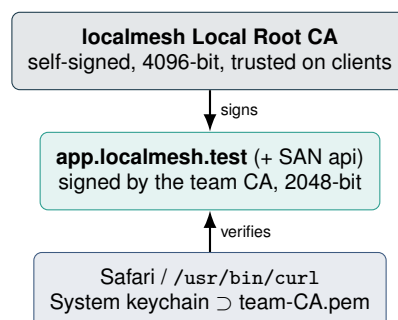


Figure 5: Private CA. Clients must trust team-CA.pem. Use /usr/bin/curl, never -k.

TLS. Observed: TLSv1.3, issuer CN=localmesh Local Root CA.

HTTP and caching. Backends return JSON and X-Backend. Cache-Control: max-age=60. ETags differ (backend-a-v1 vs backend-b-v1), so a 304 through the load balancer is not reliable. Review 1 can still show the headers. `curl -I` sends HEAD; these apps implement GET only, so HEAD on / is HTTP/1.0 501. Use GET (or `curl -D - -o /dev/null`) for status evidence.

Load balancing. nginx upstream lists both host ports, default round-robin, max_fails=1. Repeated `/api/status` must show both A and B while both processes run.

5 Implementation (what we built, not a runbook)

5.1 Private DNS

Homebrew dnsmasq on Your Mac. Wi-Fi clients set their resolver to 10.7.12.104. `.test` avoids Bonjour `.local`. Upstream servers keep the ordinary internet working while the private zone stays local.

5.2 Backends

Two Python 3 `http.server` apps bind 0.0.0.0 (required so the hypervisor can forward). Endpoints: GET / and GET /api/status. Code is in `backend-a/app.py` and `backend-b/app.py`.

5.3 Edge

nginx on Manohar's Mac. HTTP 8080 redirects to HTTPS 8443. `proxy_pass` to `http://backends`. `proxy_next_upstream` covers a dead member.

5.4 TLS

A 4096-bit team CA signs the server certificate. SAN lists both names. Every client installs the CA as a trust anchor. That is how a padlock appears without a public CA.

6 Observations and evidence

Filed under `evidence/phase1/` (see `INDEX.md`):

- DNS config and `dig` (zone, A records, SERVER line).
- Ping of both Macs; direct HTTP to :3001 and :3002 with A/B headers.
- HTTPS verbose: TLS 1.3, team CA issuer, mixed x-backend.
- Browser on `app.localmesh.test:8443` returning JSON.
- Guests listening on 0.0.0.0:3001 and :3002.
- `flow.pcap` (UDP/53 and TCP/8443).
- F1: client resolver set to 8.8.8.8 $\Rightarrow$ NXDOMAIN for `app.localmesh.test`.

Still to add before the slot if missing: Wireshark stills from the pcap (DNS, handshake, ClientHello); F2 wrong A record; F3 one backend stopped; F4 both stopped (502); F5 wrong port; `openssl verify`.

7 Failure analysis

These are required Phase 1 demonstrations. They isolate layers.

Table 2: Deliberate faults and the layer they expose.

Fault	Observation	Lesson
Wrong client DNS (8.8.8.8)	NXDOMAIN; IP ping still works	Name and reachability are independent
Wrong A record	dig succeeds, TCP/TLS to the wrong host fails	DNS is a directory
One backend stopped	Remaining X-Backend only	Edge continues; HA is incomplete without health checks in the demo path
Both backends stopped	502 from nginx	Edge is up; origin is not
Wrong destination port	Connection refused / timeout on :9999	IP $\neq$ service

Diagnosis order if faculty inject a fault: (1) name, (2) TCP to :8443, (3) TLS/issuer, (4) JSON/X-Backend.

8 Learning notes (viva)

- DNS finds an IP. TCP then builds a connection. TLS then authenticates the name.
- Clients must not skip the edge. Direct :3001/:3002 is only for proving the LAN.
- Two backends share one host IP because UTM publishes them as ports, not as extra LAN NICs.
- Trusting app.crt is wrong; trusting the CA is right.
- .local would never reach dnsmasq.

9 What is not in this repository

Detailed paste-this-next setup guides stay in the private docs/ folder (gitignored). They are operational notes, not the academic deliverable. This report, config/, backend source, and evidence/ are what Review 1 should see.

10 Conclusion

Phase 1 is a working private name, a trusted HTTPS edge, two identifiable backends, and packet-level proof of DNS, TCP, and TLS. The topology uses two Macs honestly and still presents four roles. Phase 2 (backup DNS, isolation, TTL, cutover) extends this system; it is not part of this report.

A Reference configuration

Values match a live ~/cn-team.env. Private keys are not included.

A.1 dnsmasq (excerpt)

```
listen-address=127.0.0.1,10.7.12.104
no-resolv
server=8.8.8.8
local=/localmesh.test/
host-record=app.localmesh.test,10.7.2.38
host-record=api.localmesh.test,10.7.2.38
```

A.2 nginx upstream (excerpt)

```
upstream backends {
    server 10.7.12.104:3001 max_fails=1 fail_timeout=10s;
    server 10.7.12.104:3002 max_fails=1 fail_timeout=10s;
}
```

A.3 Backend contract

GET /api/status → {"backend": "A"|"B", "status": "ok"} and header X-Backend.